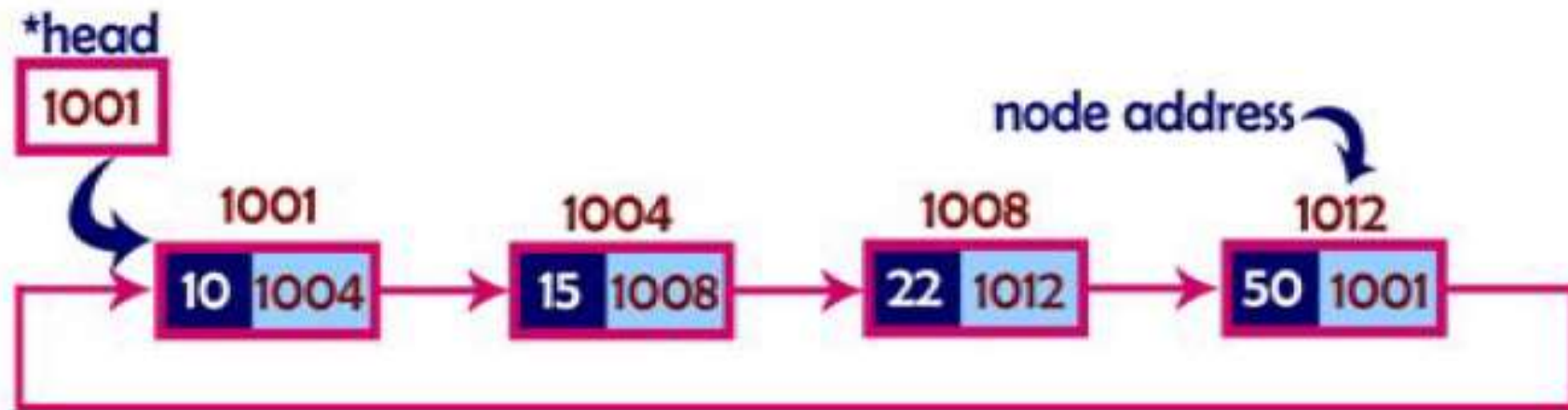


CIRCULAR LINKED LIST

Dr. Jyothi Pillai
Professor
Bhilai Institute of Technology Durg(C.G.)

Circular Linked List

- ❖ In single linked list, every node points to its next node in the sequence and the last node points to NULL.
- ❖ Circular linked list is a sequence of elements in which every element has link to its next element and last element has a link to the first element in the list.
- ❖ Circular linked list is similar to the single linked list except that the last node points to the first node in the list



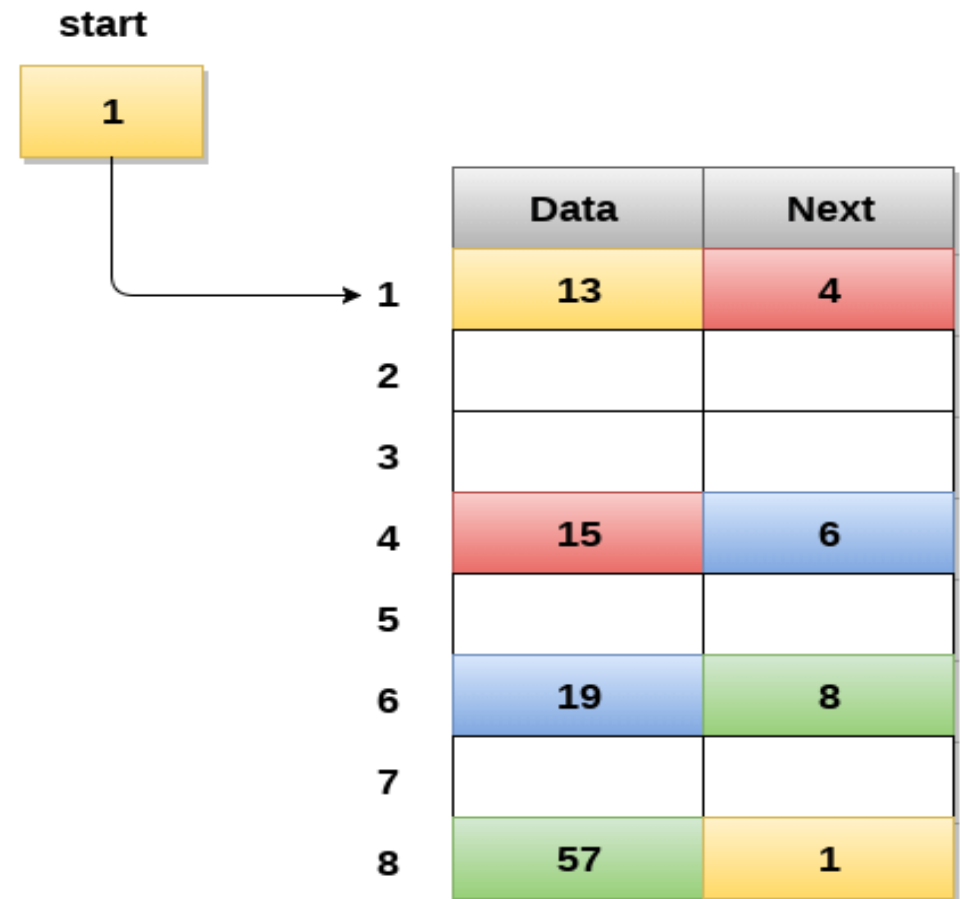
Memory Representation of circular linked list

For circular linked list in the memory; the last node of the list contains the address of the first node of the list.

For example, memory representation of a circular linked list containing marks of a student in 4 subjects.

The start or head of the list is pointing to the element with the index 1 and containing 13 marks in the data part and 4 in the next part.

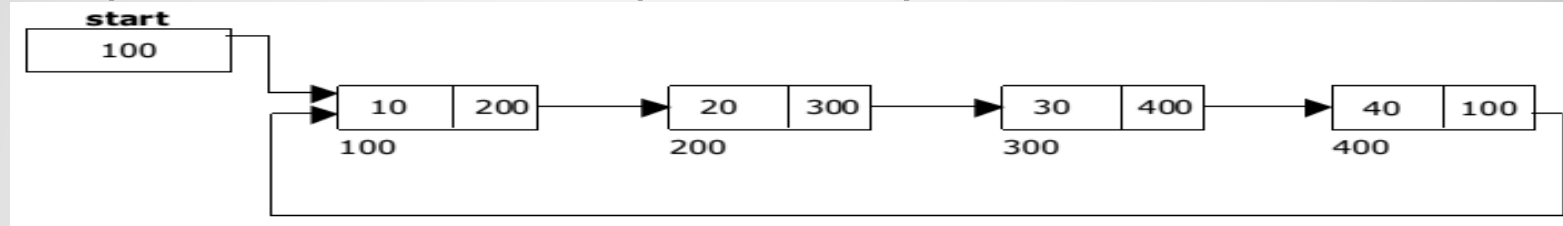
i.e. it is linked with the node that is being stored at 4th index of the list.



Types of Circular Linked lists

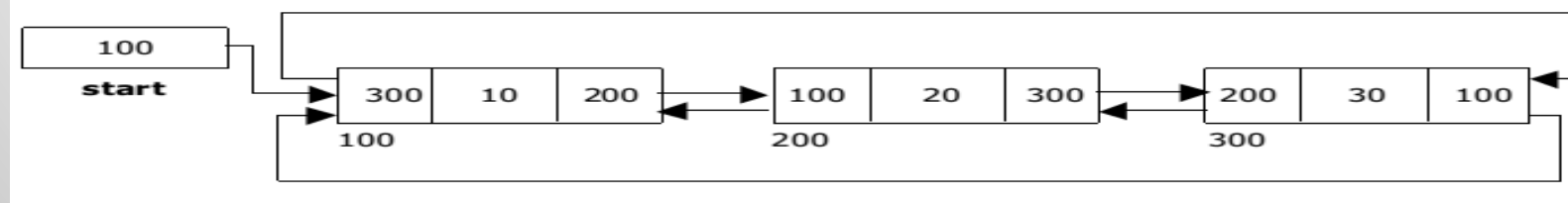
Circular Singly Linked List

- ❖ In single circular linked list, the **last** node of the list holds the address of the **first** node hence forming a circular chain.
- ❖ No null value is present in the next part of any of the nodes.



Circular Doubly Linked List

- ❖ In double circular linked list, the **next** part of Last node contains address of **first** node and **previous part** of first node has a link to **last node** making the circular in both directions.
- ❖ Circular Doubly Linked List has properties of both doubly linked list and circular linked list.



Operations for Circularly Linked Lists

In a circular linked list, following operations can be performed:-

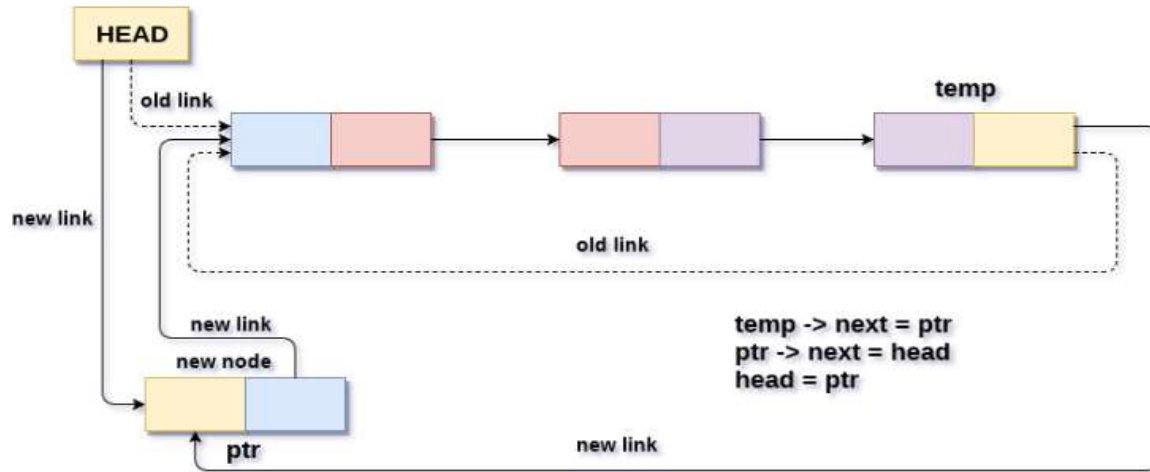
1. Insertion
2. Deletion
3. Display
4. Search

1. Insertion

In a Circular Linked list, insertion operation can be performed in three ways-

- i. Inserting At Beginning of the list
- ii. Inserting At End of the list
- iii. Inserting At Specific location in the list

i. Insertion at Beginning of the list



Algorithm

Step 1: IF PTR = NULL

Write OVERFLOW

Go to Step 11

Step 2: SET NEW_NODE = PTR

Step 3: SET PTR = PTR -> NEXT

Step 4: SET NEW_NODE -> DATA = VAL

Step 5: SET TEMP = HEAD

Step 6: Repeat Step 8 while TEMP -> NEXT != HEAD

Step 7: SET TEMP = TEMP -> NEXT

Step 8: SET NEW_NODE -> NEXT = HEAD

Step 9: SET TEMP $\rightarrow$ NEXT = NEW NODE

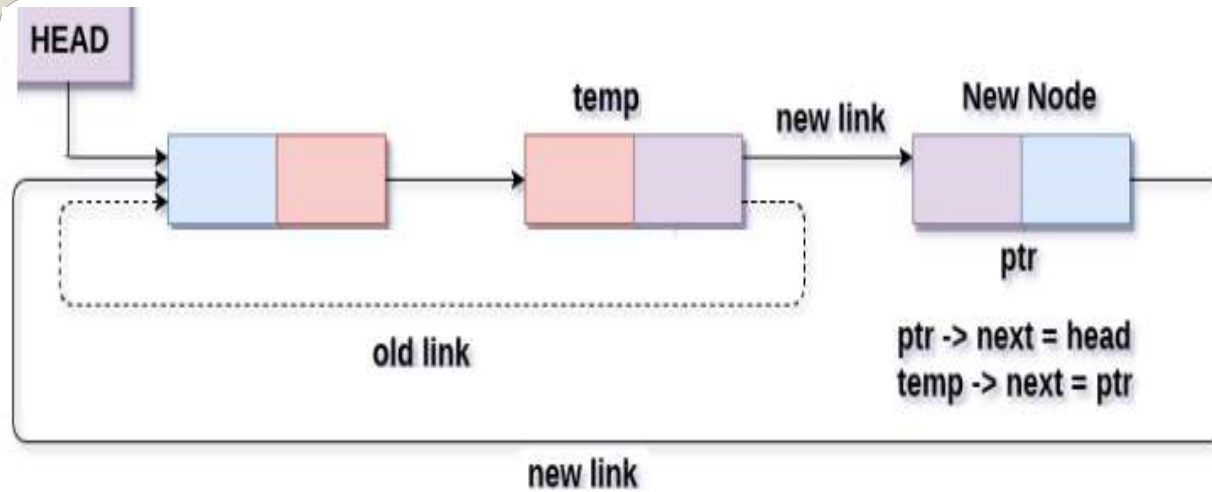
Step 10: SET HEAD = NEW NODE

Step 11: EXIT

C program

```
void beg_insert(int item)
{struct node *ptr = (struct node*)malloc(sizeof(struct node));
  struct node *temp;
  if(ptr == NULL)
    { printf("\nOVERFLOW"); }
  else
    { ptr -> data = item;
      if(head == NULL)
        { head = ptr; ptr -> next = head; }
      else
        { temp = head;
          while(temp->next != head)
            temp = temp->next;
          ptr->next = head;
          temp -> next = ptr;
          head = ptr; }
      printf("\nNode Inserted\n");
    }
}
```

ii. Insertion at End of the list



Algorithm

Step 1: IF PTR = NULL
Write OVERFLOW
Go to Step 1

Step 2: SET NEW_NODE = PTR

Step 3: SET PTR = PTR -> NEXT

Step 4: SET NEW_NODE -> DATA = VAL

Step 5: SET NEW_NODE -> NEXT = HEAD

Step 6: SET TEMP = HEAD

Step 7: Repeat Step 8 while TEMP -> NEXT != HEAD

Step 8: SET TEMP = TEMP -> NEXT

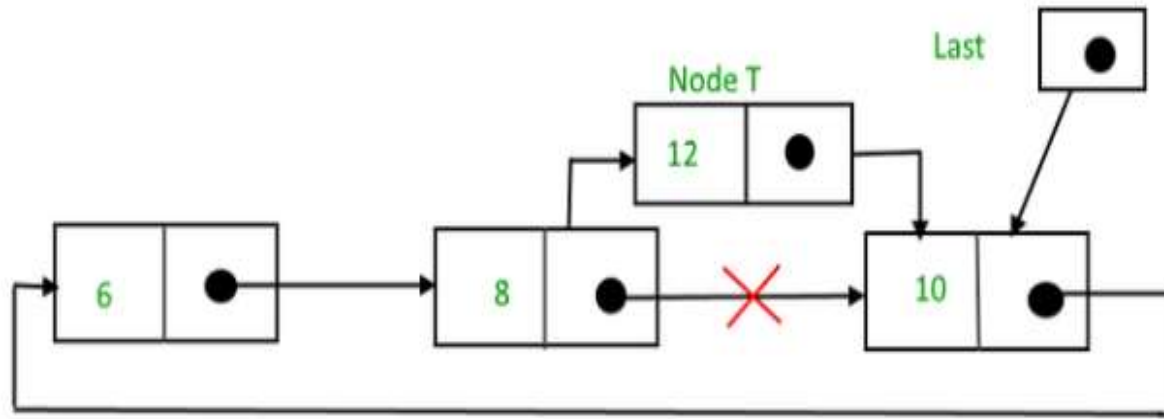
Step 9: SET TEMP -> NEXT = NEW_NODE

Step 10: EXIT

C program

```
void lastinsert(struct node*ptr, struct node *temp, int item)
{ ptr = (struct node *)malloc(sizeof(struct node));
  if(ptr == NULL)
  { printf("\nOVERFLOW\n"); }
  else
  { ptr->data = item;
    if(head == NULL)
    { head = ptr;
      ptr -> next = head; }
    else
    { temp = head;
      while(temp -> next != head)
      { temp = temp -> next; }
      temp -> next = ptr;
      ptr -> next = head;
    }
  }
}
```


iii. Insertion At Specific location in the list



Algorithm

Step 1: START

Step 2: Store the data to create linked list.

Step 3 Store the element to be at any position of list.

Step 4: Store the position and start the counter.

Step 5 Check if head==null not possible else goto step 6.

Step 6: Store data at stored position.

Step 7: Create new link for data stored and point towards next position.

Step 8: Change the link address of next and previous node where new node is entered.

Step 9: STOP

C program

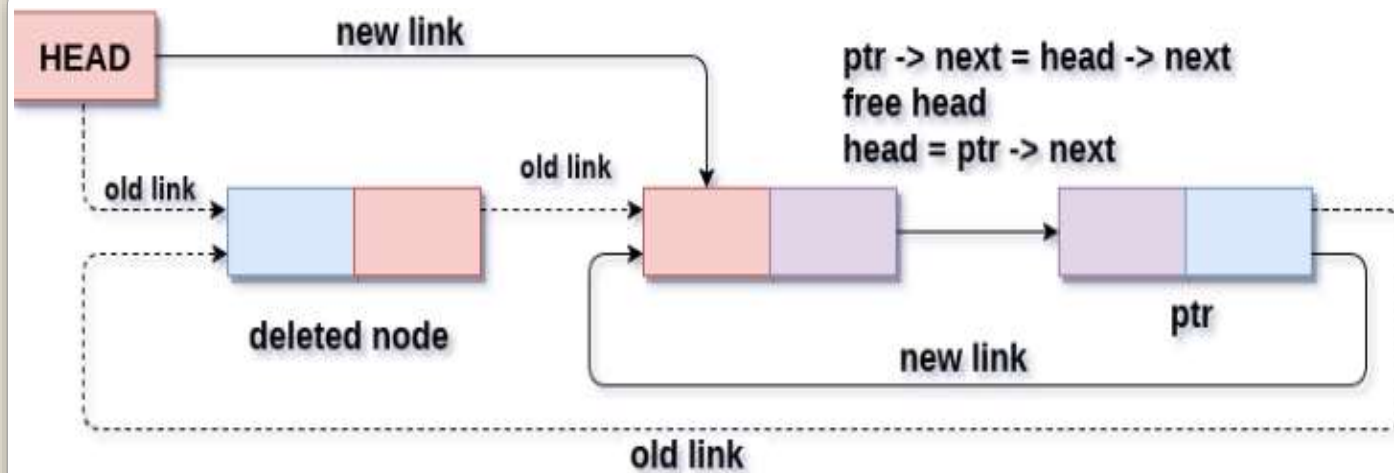
```
void add_node_afternode(struct node** root, *temp, *qtr)
void ins_at_pos()
{ struct node *ptr;
  int c = 1, pos, count = 1;
  y = (struct node*)malloc(sizeof(struct node));
  if (head == NULL)
    {printf("cannot enter an element at this place"); }
  printf("\n Enter the data for particular position:");
  scanf("%d", &y->data);
  printf("\n Enter the position to be inserted:");
  scanf("%d", &pos);
  x = head; ptr = head;
  while (ptr->link != head)
    { count++; ptr = ptr->link; }
  count++;
  if (pos > count)
    { printf("OUT OF BOUND");
      return; }
  while (c < pos)
    { z = x; x = x->link;
      c++; }
  y->link = x;
  z->link = y; }
```

2. Deletion

In a Double Linked list, deletion operation can be performed in three ways -

- i. Deleting from Beginning of the list
- ii. Deleting from End of the list
- iii. Deleting a Specific Node

1. Deletion from Beginning of the list



Algorithm

STEP 1: IF HEAD = NULL
Write UNDERFLOW
Go to Step 8

Step 2: SET PTR = HEAD

Step 3: Repeat Step 4 while PTR → NEXT != HEAD

Step 4: SET PTR = PTR → next

Step 5: SET PTR → NEXT = HEAD → NEXT

Step 6: FREE HEAD

Step 7: SET HEAD = PTR → NEXT

Step 8: EXIT

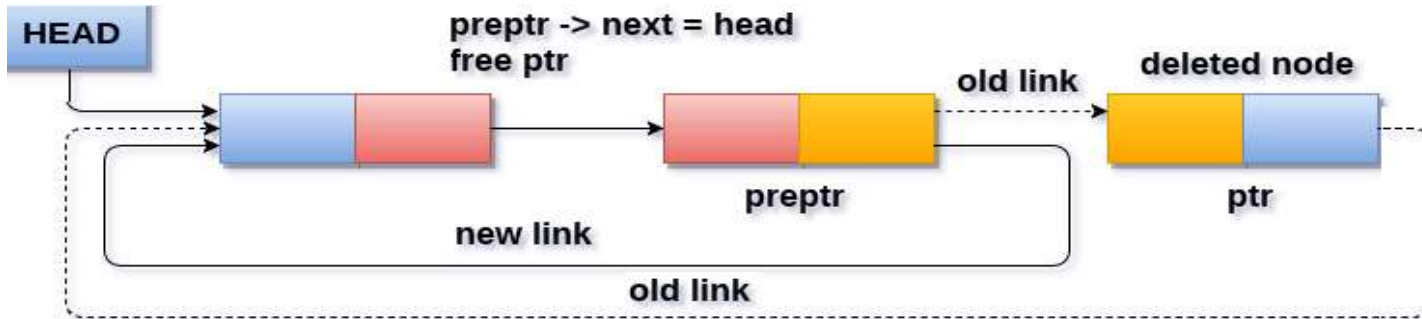
C program

```
void beg_delete()
{ struct node *ptr;
  if(head == NULL)
    { printf("\nUNDERFLOW\n"); }
  else if(head->next == head)
    { head = NULL;
      free(head);
      printf("\nNode Deleted\n");
    }
  else
    { ptr = head;
      while(ptr->next != head)
        ptr = ptr->next;
      ptr->next = head->next;
      free(head);
      head = ptr->next;
      printf("\nNode Deleted\n");
    }
}
```

2. Deletion from End of the list

There are three scenarios of deleting a node in circular singly linked list at the end-

- list is empty
- list contains single element
- list contains more than one element



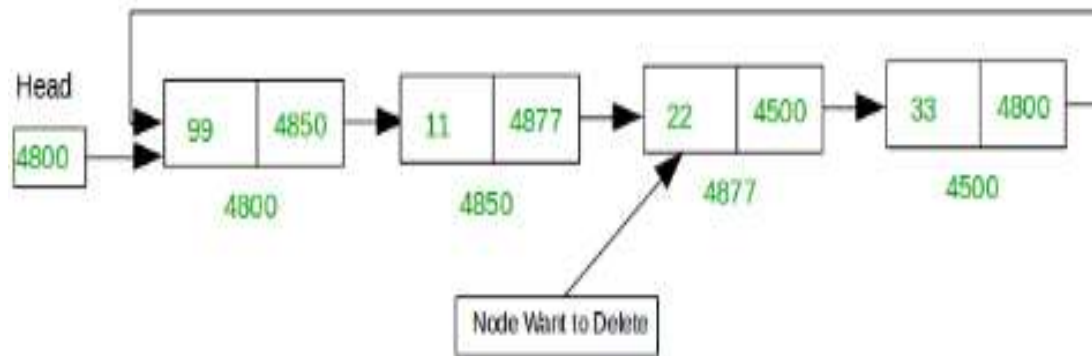
Algorithm

- Step 1: IF HEAD = NULL
Write UNDERFLOW
Go to Step 8
- Step 2: SET PTR = HEAD
- Step 3: Repeat Steps 4 and 5 while PTR->NEXT != HEAD
- Step 4: SET PREPTR = PTR
- Step 5: SET PTR = PTR -> NEXT
- Step 6: SET PREPTR -> NEXT = HEAD
- Step 7: FREE PTR
- Step 8: EXIT

C program

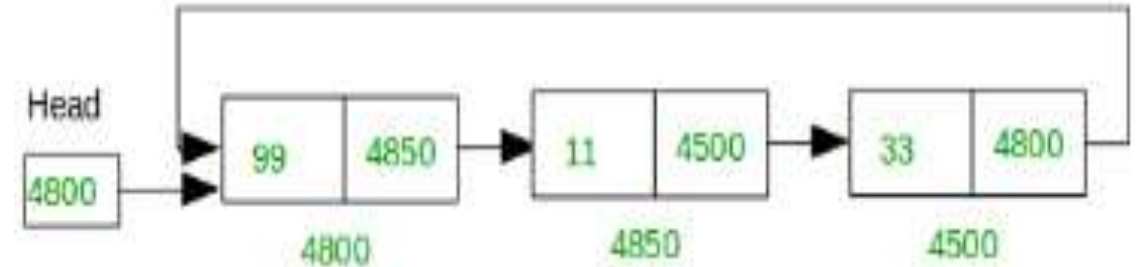
```
void last_delete()
{ struct node *ptr, *preptr;
  if(head==NULL)
    {printf("\nUNDERFLOW\n"); }
  else if (head->next == head)
    { head = NULL;
      free(head);
      printf("\nNode Deleted\n"); }
  else
    { ptr = head;
      while(ptr ->next != head)
        {preptr=ptr;
          ptr = ptr->next;
        }
      preptr->next = ptr -> next;
      free(ptr);
      printf("\nNode Deleted\n");
    }
}
```

3. Deletion from Specified position of the list



Algorithm

Step 1: START
Step 2: Store the element to be deleted.
Step 3: Store the position.
Step 4: Initialize counter c=1
Step 5: Check while (c<pos)
Step 6: Count the position of data to be deleted
 y = x; x = x->link;
 c++;
Step 7: Use free() to delete
 y->link = x->link;
 free(x);
Step 8: STOP



C program

```
void del_at_pos()
{
    if (head == NULL)
        printf("\n List is empty");
    else
    {
        int c = 1, pos;
        printf("\n Enter the position to be deleted:");
        scanf("%d", &pos);
        x = head;
        while (c < pos)
        {
            y = x;
            x = x->link;
            c++;
        }
        y->link = x->link;
        free(x);
    }
}
```

3. Display

Algorithm

Step 1: Set PTR = HEAD

Step 2: If HEAD != NULL Then

Repeat step 3 to 4 while PTR != NULL

Step 3: Print PTR → data

Step 4: PTR = PTR → next

Step 5: Exit

C program

```
void display()
{ struct node *ptr = head;
  printf("\n[ ");
  //start from the beginning
  if(head != NULL)
  { while(ptr->next != ptr)
    { printf("(%d,%d) ", ptr->key, ptr->data);
      ptr = ptr->next; }
    }
  printf(" ]");
}
```

4. Search

Algorithm

Step 1: SET PTR = HEAD

Step 2: Set I = 0

STEP 3: IF PTR = NULL

 WRITE "EMPTY LIST"

 GOTO STEP 8

STEP 4: IF HEAD → DATA = ITEM

 WRITE i+1 RETURN

STEP 5: REPEAT STEPS 5-7 UNTIL PTR->next != HEAD

STEP 6: if ptr → data = item

 WRITE i+1

 RETURN

STEP 7: I = I + 1

STEP 8: PTR = PTR → NEXT

STEP 9: EXIT

C program

```
void search()
{ struct node *ptr;
  int item,i=0,flag=1;
  ptr = head;
  if(ptr == NULL)
    {printf("\nEmpty List\n"); }
  else
    { printf("\nEnter item to search?\n");
      scanf("%d",&item);
      if(head ->data == item)
        { printf("item found at location %d",i+1);
          flag=0; return; }
      else
        { while (ptr->next != head)
            { if(ptr->data == item)
                {printf("item found at location %d ",i+1);
                  flag=0; return; }
              else
                { flag=1; }
              i++;
              ptr = ptr -> next; }
          if(flag != 0)
            { printf("Item not found\n"); return; }
        }
    }
```

Advantages of Circular Linked Lists

- ***Any node can be a starting point.*** Traverse the whole list starting from any point and stop when the first visited node is visited again.
- ***Useful for implementation of a queue.*** No need to maintain two pointers for front and rear if circular linked list is used. Maintain a pointer to the last inserted node and the front can always be obtained as next of last.
- ***Circular lists are useful in applications to repeatedly go around the list.***
For example, when multiple applications are running on a PC, it is common for the operating system to put the running applications on a list and then cycle through them, giving each of them a slice of time to execute, and then making them wait while the CPU is given to another application.
It is convenient for the operating system to use a circular list so that when it reaches the end of the list it can cycle around to the front of the list.
- ***Circular Doubly Linked Lists are used for the implementation of advanced data structures*** like the Fibonacci Heap.
- ***Implementing a circular linked list*** can be relatively easy compared to other more complex data structures like trees or graphs.

Disadvantages of circular linked list

- Compared to singly linked lists, circular lists are more complex.
- Reversing a circular list is more complicated than singly or doubly reversing a circular list.
- It is possible for the code to go into an infinite loop if it is not handled carefully.
- It is harder to find the end of the list and control the loop.
- Although circular linked lists can be efficient in certain applications, their performance can be slower than other data structures in certain cases, such as when the list needs to be sorted or searched.
- Circular linked lists don't provide direct access to individual nodes

Applications of circular linked lists

- Multiplayer games use this to give each player a chance to play.
- A circular linked list can be used to organize multiple running applications on an operating system. These applications are iterated over by the OS.
- Circular linked lists can be used in resource allocation problems.
- Circular linked lists are commonly used to implement circular buffers,
- Circular linked lists can be used in simulation and gaming.

THANKYOU